04.02.2022 Examen PCLP - Grupa 311CA **Student** GRANCSA ROBERT

Observații generale

1. **Nu** se vor declara variabile globale. Se acceptă macrodefiniții sau declarația de constante globale.
2. Pentru primele două probleme, soluțiile corecte cu nerespectarea condițiilor specificate conduce la acordarea unui punctaj parțial pentru acestea.

[5p] 1. Calul nărăvaș. Un cal pe o tablă de șah, fiind așezat într-o anumită poziție, poate acoperi, prin mutări succesive toată tabla de șah. Astfel, luăm în considerare o matrice pătratică `NxN` care conține valorile de la `1` la N^2. Valorile din matrice reprezintă pozițiile succesive ale calului pe tabla de șah, acoperind toate pătrățelele acesteia, punctul de plecare al calului fiind valoarea `1` din matrice. Ne dorim să stabilim un astfel de traseu reprezintă sau nu o acoperire corectă a unei table de șah generalizate (de dimensiuni `NxN`) prin deplasări ale calului (conform regulilor de deplasare ale calului pe o tablă de șah, în forma literei L, cel mai mic L posibil), pornind din poziția inițială.

Dintr-un fișier text, al cărui nume se citește de la tastatură (cu extensie sau fără), se citesc mai multe seturi de forma `N` și matricea de valori specificată. În implementare se va folosi o singură matrice care este realocată dinamic pentru fiecare set, după citirea numărului `N`. Rezultatul pentru fiecare set se va scrie pe o linie într-un fișier text cu extensia `.out` (dacă fișierul de intrare avea extensie, aceasta se înlocuiește cu `.out`, dacă nu avea, doar se adaugă extensia `.out`), o linie având forma următoare (2 variante posibile):

a. va conține pe o linie răspunsul `DA`, dacă matricea este un traseu corect, sau
b. va conține pe o linie răspunsul `NU`, dacă matricea nu reprezintă un traseu corect iar în apoi se va scrie pasul la care se află calul, precum și poziția lui pe tablă (linia, coloana).

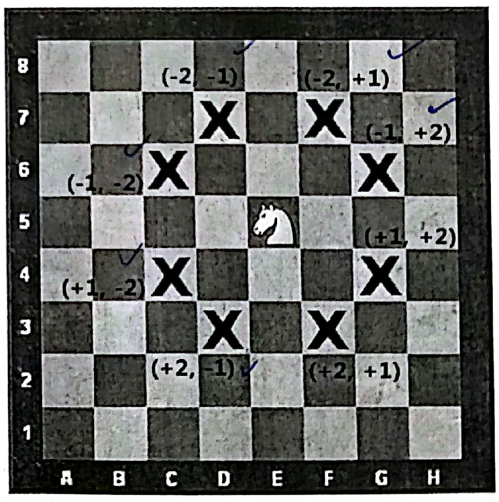
Cerință obligatorie: pentru codificarea poziției calului pe tabla de șah se va folosi o structură definită de voi.

Exemplu:

traseu.in

```
8
 1 38 55 34  3 36 19 22
54 47  2 37 20 23  4 17
39 56 33 46 35 18 21 10
48 53 40 57 24 11 16  5
59 32 45 52 41 26  9 12
44 49 58 25 62 15  6 27
31 60 51 42 29  8 13 64
50 43 30 61 14 63 28  7
5
25  6  9 16 23
24 15  2  7 10
 5  8 11 22 13
20  1 14  3 19
17  4 18 12 21
```

Săritura calului



traseu.out

```
DA
NU 16 1 4
```

Explicație: Pentru exemplul 1, calul pleacă din poziție (8,A) - cifra 1, merge la (2,C) - cifra 2, apoi la (8,E) - cifra 3, etc.

[5p] 2. Șiruri cu... perioadă. Un șir de caractere are perioada `k` dacă poate fi format prin concatenarea succesivă (de cel puțin o dată) a unui șir de lungime `k`. De exemplu, șirul "`abcabcabcabc`" are perioada 3 deoarece el poate fi scris ca o repetiție de 4 secvențe de "`abc`". Acesta poate avea și perioada 6 (două repetiții ale șirului "`abcabc`") sau perioada 12 (o singură apariție a șirului "`abcabcabcabc`").

Pe noi ne interesează cea mai mică perioadă a unui șir. Problema este următoarea: avem un fișier binar care conține structuri de forma `{lungime_sir, perioada, sir}`. Fișierul a fost greșit construit și perioada are valoarea 0 pentru fiecare structură din fișier, șirul și lungimea lui, fiind corect date. Sarcina noastră este să scriem un program care prelucrează fișierul și afișează informațiile corectate din aceasta. Programul poate primi **în linie de comandă** parametri după cum urmează:

a. numele fișierului de intrare și apoi niciun parametru - programul doar corectează fișierul;
b. numele fișierului de intrare și apoi parametrul `showall` - programul corectează fișierul nostru și afișează toate informațiile din fișier.
c. numele fișierului de intrare, apoi parametrul `showpos`, urmat de un șir de numere naturale care reprezintă numere de ordine din fișier (nu neapărat sortat crescător) - programul corectează fișierul nostru și afișează informațiile din fișier doar pentru structurile cu numărul de ordine indicat (numerotarea pozițiilor se face de la 1).

Cerințe obligatorii:

a. fișierul nu se încarcă în memorie;
b. la afișare, se va scrie pe ecran, pentru fiecare structură din fișier, pe o linie, perioada minimă și secvența care se repetă corespunzător separate printr-un spațiu.

Exemplu:
date.in (reprezentare în format citibil):
`{8,0,"HoHoHoHo"},{12,0,"HiHaHoHiHaHo"},{6,0,"CinCin"},{4,0,"PCLP"},{5,0,"aaaaa"}`
comanda
`./a.out date.in showpos 5 3 2 1 4`
va avea ca efect:
- fișier corectat:
`{8,2,"HoHoHoHo"},{12,6,"HiHaHoHiHaHo"},{6,3,"CinCin"},{4,4,"PCLP"},{5,1,"aaaaa"}`
- afișare:

```
1 a
3 Cin
6 HiHaHo
2 Ho
4 PCLP
```

[1p] Bonus. Scrieți o funcție `unsigned rightrot(unsigned x, int n)` care returnează valoarea întregului `x` rotit la dreapta pe biți cu `n` poziții [0,5p].
Apoi scrieți o funcție care calculează distanța Hamming între numărul `x` și o rotire posibilă a lui (`y`): `int hamming(unsigned x, unsigned y)`. În teoria informației, distanța Hamming dintre două șiruri de lungime egală este numărul de poziții ale căror simboluri corespunzătoare sunt diferite. De exemplu distanța Hamming dintre `1011101` și `1001001` este `2` [0,5p].